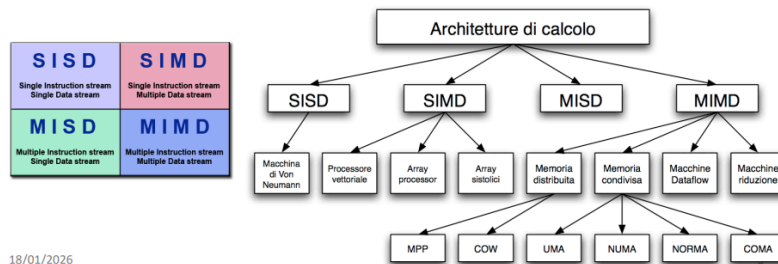


Sistemi per il calcolo ad alte prestazioni

La tassonomia di Flynn: classificare l'hardware

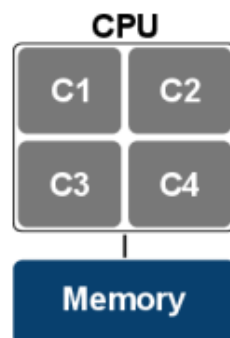
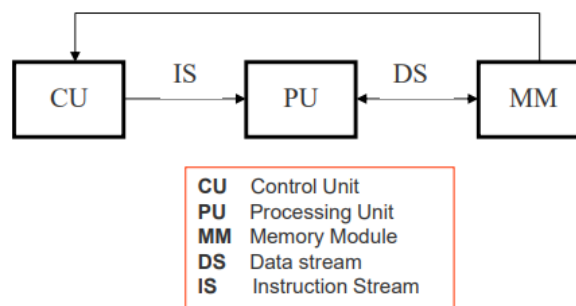
Per orientarsi tra i vari tipi di computer, il modello più utilizzato è la Tassonomia introdotta da M.J. Flynn nel 1966. Questa classificazione si basa su due dimensioni indipendenti: il **flusso di istruzioni (Instruction Stream)** e il **flusso di dati (Data Stream)**, che possono essere singoli (single) o multipli (multiple).



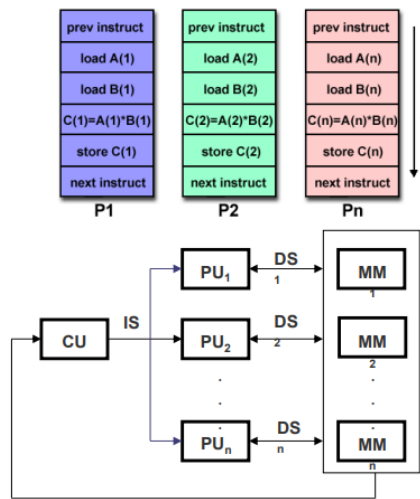
18/01/2026

Da questa matrice nascono quattro categorie principali:

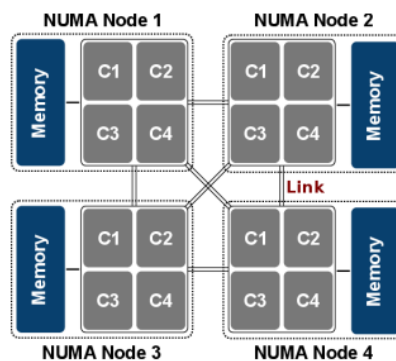
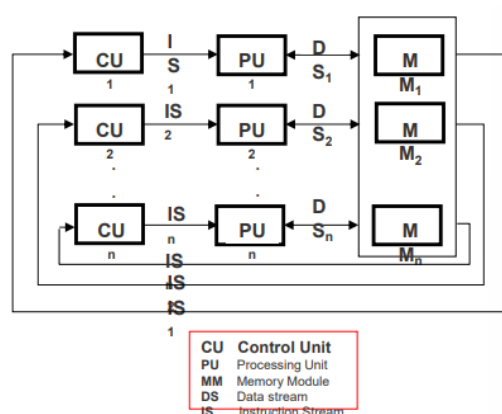
- **SISD (Single Instruction, Single Data)**: È la classica architettura di Von Neumann. Un'unica unità di controllo invia un solo flusso di istruzioni a un'unità di calcolo, che opera su un singolo flusso di dati. Rappresenta i vecchi processori sequenziali.



- **MISD (Multiple Instruction, Single Data)**: Più flussi di istruzioni operano sullo stesso dato. È un'architettura rara, usata per lo più in sistemi ridondanti per la tolleranza ai guasti (es. computer di volo).
- **SIMD (Single Instruction, Multiple Data)**: Un grande passo verso il parallelismo. I sistemi SIMD possiedono un'unica unità di controllo centrale che impartisce la *stessa istruzione* a molteplici unità di calcolo (Processing Units), le quali la eseguono contemporaneamente su *dati diversi*.



- **MIMD (Multiple Instruction, Multiple Data):** Ogni processore è indipendente. Ogni core ha la propria unità di controllo ed esegue il proprio flusso di istruzioni su un set di dati distinto. Quasi tutti i moderni sistemi di calcolo parallelo ricadono in questa categoria.



Approfondimento sui sistemi SIMD (Istruzioni vettoriali)

Nei sistemi SIMD, l'obiettivo è applicare la stessa operazione matematica a un "vettore" di dati contemporaneamente. Ci sono due tipologie principali:

- **Processori vettoriali:** macchine storiche dotate di matrici (array) di unità di calcolo che condividono l'unità di controllo, dove ogni unità ha la propria memoria. Esempi storici includono il CRAY-1 degli anni '70 e il Connection Machine 2 (CM2) degli anni '80.
- **Istruzioni vettoriali (Moderne):** i processori attuali implementano il parallelismo SIMD direttamente all'interno dei core integrando istruzioni vettoriali a fianco di quelle scalari normali. Le istruzioni vettoriali specificano una particolare operazione che deve essere eseguita su un determinato insieme di operandi detto vettore. Le unità funzionali che eseguono istruzioni vettoriali sfruttano il pipelining

per eseguire la stessa operazione su tutte le coppie di operandi.

Esempi di queste estensioni nei processori Intel includono SSE (vettori a 128 bit), AVX (256 bit) e AVX-512 (512 bit).

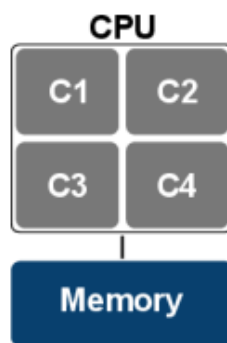
Il Limite di Flynn: Memoria Condivisa vs Distribuita

La tassonomia di Flynn ha un limite concettuale: non distingue bene come i processori accedano alla memoria. Nel mondo MIMD, la gestione della memoria è lo spartiacque fondamentale tra le architetture.

Architetture a Memoria Condivisa (Shared Memory)

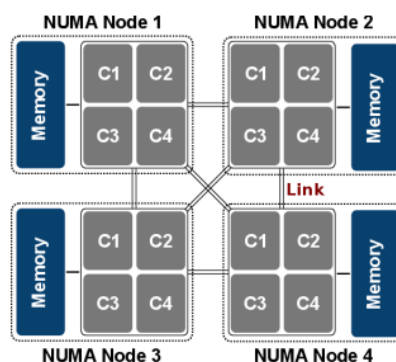
In questi sistemi, tutti i processori accedono alla memoria RAM come a uno spazio di indirizzamento globale e condiviso; se una CPU modifica una variabile, la modifica è immediatamente visibile alle altre. Esistono due implementazioni principali:

1. **UMA (Uniform Memory Access):** conosciuto anche come SMP (Symmetric Multi Processors). Qui tutti i processori accedono alla memoria con lo stesso tempo di accesso. Il problema? All'aumentare dei core, il bus di memoria diventa un collo di bottiglia e il sistema non scala.



1. **NUMA (Non-Uniform Memory Access):** per risolvere il collo di bottiglia, i moderni sistemi multisocket assegnano a ogni CPU (o nodo NUMA) una propria porzione di memoria locale. L'hardware si occupa di unire queste memorie locali in un unico spazio globale, permettendo a una CPU di indirizzare sia la propria RAM che quella delle altre CPU. Tuttavia, *il tempo di accesso non è uniforme*: leggere dalla RAM collegata al proprio socket è molto veloce, mentre leggere dalla RAM di un altro socket richiede l'attraversamento di un link di interconnessione, introducendo ritardo.

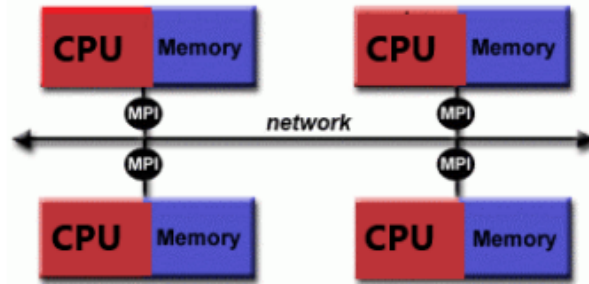
Il comando `numactl`: in ambiente Linux, il programmatore può ottimizzare le prestazioni controllando dove il sistema operativo piazza i calcoli e i dati. Il comando `numactl -H` mostra la topologia del sistema (i core disponibili per ogni nodo NUMA e le distanze relative). Usando flag come `--cpubind` e `--membind`, possiamo forzare un processo a usare solo una specifica CPU e la memoria ad essa vicina, minimizzando la latenza.



Architetture a Memoria Distribuita (Distributed Memory)

In questo caso, creiamo un cluster collegando fisicamente più computer (chiamati nodi). Ogni CPU possiede la sua memoria locale esclusiva, totalmente isolata e invisibile agli altri nodi. Per collaborare, i nodi devono spedirsi messaggi attraverso una rete (generalmente usando lo standard MPI).

- **Vantaggi:** costi contenuti (si può usare hardware *commodity*, ovvero server standard di mercato) e scalabilità teoricamente illimitata (basta aggiungere nodi per avere più core e più RAM).
- **Svantaggi:** scrivere il software è molto più complesso, poiché il programmatore deve orchestrare manualmente lo scambio di dati. Inoltre, l'accesso a un dato remoto dipende dalla rete ed è estremamente più lento rispetto alla memoria locale.



Il tempo T per inviare un messaggio si calcola approssimativamente come la somma della **latenza** (L , il tempo tecnico di gestione e propagazione del segnale nei cavi e negli switch) e del **tempo di trasmissione** (T_{tr} , che dipende dalla dimensione dei dati divisa per la lunghezza di banda).

Reti ad alte prestazioni per HPC (come Intel OmniPath o Infiniband EDR) offrono larghezze di banda spaventose (12 GB/s) e latenze irrisorie ($O(1)$ microsecondo) se paragonate alla comune rete Ethernet Gigabit, che ha una banda di soli 0.1 GB/s e latenze dell'ordine di grandezza di $O(100)$ microsecondi.

Storage per l'HPC

Quando migliaia di processori elaborano dati contemporaneamente, il classico disco rigido non basta. Nei cluster HPC si implementa il concetto di **Storage Area Network (SAN)**: una rete ad alta velocità dedicata esclusivamente a connettere i server di calcolo a potenti armadi di dischi (Disk Array). Le SAN offrono ridondanza, scalabilità e prestazioni altissime, sebbene siano costose e complesse da gestire.

I dati non sono tutti uguali, per cui si usano strategie di **Tiering** (gerarchie di archiviazione):

- **Hot Tier (Online):** dati usati spessissimo. Capacità ridotta (sotto il 10% del totale) ma velocità estreme. Usa dischi SSD capaci di leggere a centinaia di MB/s.
- **Warm Tier (Nearline):** dati ad accesso saltuario, memorizzati su classici e capienti dischi meccanici HDD.
- **Cold Tier (Offline):** Giganteschi archivi per dati usati raramente (occupano l'80% dello storage). Si usano sistemi a nastro magnetico (TAPE), lenti ma capacissimi (es. cartucce da 30 TB).

Per gestire lo storage a livello logico servono *File System Paralleli* (come **GPFS** di IBM o **Lustre**). Invece di salvare un grosso file su un singolo disco creandovi un collo di bottiglia, questi file system spezzettano il file in blocchi e li sparpagliano su moltissimi dischi diversi (*striping*). In questo modo, quando il cluster richiede quel file, centinaia di dischi lavorano in parallelo per fornirlo. Inoltre, separano la gestione dei dati (i file veri e propri) dalla gestione dei metadati (nomi, permessi, posizioni) per evitare sovraccarichi strutturali.

	Bandwidth (GB/s)	Latency (microsec.)	Costo scheda
Intel OmniPath e Infiniband EDR	12	$O(1)$	
10GbEthernet	0.9	$O(10)$	
GbEthernet	0.1	$O(100)$	

I Sistemi Ibridi

Oggi, per superare i limiti di scalabilità spaziale ed energetica, tutti i supercomputer più potenti adottano un'architettura **ibrida**.

Cosa significa? Si costruiscono enormi reti a scambio di messaggi (MIMD a memoria distribuita), dove ogni singolo "nodo" è a sua volta un potentissimo computer a memoria condivisa (spesso architetture NUMA multicore). A completare l'ibridazione, le ultime generazioni di sistemi affiancano alle CPU potenti **acceleratori GPU**, connessi tramite bus di I/O (es. PCI), dotati di una loro memoria autonoma e programmati tramite linguaggi specifici come CUDA.

Casi di Studio: Leonardo e HPC UniPR

Un esempio lampante di architettura ibrida all'avanguardia è **Leonardo**, ospitato al CINECA e posizionato ai vertici mondiali:

- È diviso in due macro-sezioni: 3456 nodi "booster" progettati per la massima potenza computazionale (ogni nodo unisce una CPU Intel Xeon a 32 core a ben 4 potentissime GPU Nvidia Ampere) e 1536 nodi "data-centric" senza GPU, dotati però di 2 CPU Intel Sapphire Rapids da 56 core e mezzo terabyte di RAM velocissima per nodo.
- La rete che unisce tutto questo è una Nvidia Mellanox Dragon FI+ capace di 200 Gbit/s di banda.
- Il comparto storage vanta un Fast tier Flash da 5.6 Petabyte (con velocità mostruose fino a 1400 GB/s) affiancato da un Capacity Tier da 106 Petabyte.

Un'architettura simile, seppur su scala locale, è presente nel data center **HPC dell'Università di Parma**:

- Il cluster di calcolo integra nodi CPU (misti tra Broadwell, Skylake e AMD) e nodi GPU equipaggiati con schede NVIDIA (P100, V100 e le performanti A100 da 135 Tflops ciascuna).
- Le comunicazioni viaggiano su rete OmniPath (100Gb/s di banda e latenza bassissima di 100ns).
- Lo storage GPFS è suddiviso in base al caso d'uso: una porzione "hot" su dischi ibridi HDD+SSD (46 TB) destinata ai programmi in esecuzione intensiva (la directory `/hpc/scratch`), affiancata da tier "warm" per le cartelle utente e "cold" (176 TB) per l'archiviazione a lungo termine.